

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	Haresh Kumar N L	Reg. No.:	192425009
Section / Batch:	AI&ML	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

1) Debugging Selection Sort Code (Minimum Index Not Reset Properly) The following Selection Sort implementation fails because the minimum index is initialized only once outside the outer loop, causing comparisons in later passes to use an outdated reference. Carefully analyze why the algorithm loses track of the current unsorted portion. Apply systematic debugging to reset the minimum index correctly in every pass. Optimize the implementation by avoiding unnecessary swaps when the selected minimum is already in place. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    min_idx = 0 # ERROR
    for i in range(n):
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Code of Conduct

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Answer:

Given Code:

```
def selection_sort(arr):
    n = len(arr)
    min_idx = 0 # ERROR
    for i in range(n):
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Why does the algorithm fail?

The error is that min_idx is initialized only once before the outer loop.

min_idx = 0

Since min_idx is not reset inside the outer loop:

- After the first pass, it still stores the index of the previous minimum.
- During the next pass, comparisons are made with an outdated index.
- As a result, the algorithm may compare elements from the already sorted part instead of the current unsorted part.
- This produces incorrect sorting.

Debugging:

Step 1: Move

min_idx = i

inside the outer loop.

Step 2: Search only in the unsorted portion.

for j in range(i + 1, n):

Step 3: Update the minimum index whenever a smaller element is found.

if arr[j] < arr[min_idx]:

min_idx = j

Step 4: Swap only if needed.

Instead of

arr[i], arr[min_idx] = arr[min_idx], arr[i]

use

if min_idx != i:

arr[i], arr[min_idx] = arr[min_idx], arr[i]

This avoids unnecessary swaps.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
numbers = [64, 25, 12, 22, 11]
print("Original Array:", numbers)
sorted_array = selection_sort(numbers)
print("Sorted Array:", sorted_array)
```

```
= RESTART: C:/Users/hk838/Downloads/S.
e.py
Original Array: [64, 25, 12, 22, 11]
Sorted Array: [11, 12, 22, 25, 64]
```

Time Complexity Analysis

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Explanation

- The outer loop runs $n - 1$ times.
- The inner loop scans the remaining unsorted elements.
- Total comparisons:

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$$

which is $O(n^2)$.

Although unnecessary swaps are avoided, the number of comparisons remains the same, so the time complexity does not change.

Space Complexity

- Space Complexity: $O(1)$ (in-place sorting)
- Only a few extra variables (n , i , j , min_idx) are used.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____